

Introduction to Spring Boot

A Beginner-Friendly Guide to Building Modern Java Applications

Author: Luv, Fatma

Table of Contents

1. Introduction to Spring Boot
2. Understanding the Spring Framework
3. Setting Up the Development Environment
4. Creating Your First Spring Boot Application
5. Understanding Project Structure
6. Dependency Injection and Beans
7. Building REST APIs
8. Working with Databases
9. Spring Data JPA
10. Validation and Exception Handling
11. Spring Boot Security Basics
12. Configuration and Properties
13. Testing Spring Boot Applications
14. Deploying Spring Boot Applications
15. Best Practices in Spring Boot
16. Real-World Project Ideas
17. Conclusion

Introduction

Modern software applications require speed, scalability, security, and maintainability. Spring Boot is one of the most popular Java frameworks used to build professional backend systems and enterprise applications. Spring Boot simplifies Java development by reducing configuration complexity and providing ready-to-use tools for building web applications, APIs, and microservices.

This ebook introduces the core concepts of Spring Boot through practical examples and step-by-step explanations designed especially for beginners.

Chapter 1 — Introduction to Spring Boot

Spring Boot is an extension of the Spring Framework that makes Java development faster and easier.

Why Use Spring Boot?

Spring Boot provides:

- Fast application setup
- Minimal configuration
- Embedded servers
- Production-ready features

- Easy database integration
- Powerful dependency management

Applications Built with Spring Boot

- REST APIs
- E-commerce systems
- Banking applications
- Microservices
- Enterprise platforms
- Cloud applications

Chapter 2 — Understanding the Spring Framework

Spring Boot is built on top of the Spring Framework.

Core Concepts of Spring

Concept	Description
IoC	Inversion of Control
DI	Dependency Injection
Beans	Managed Java objects
MVC	Model View Controller

What Spring Boot Solves

Traditional Spring applications require heavy XML and configuration files. Spring Boot reduces this complexity using automatic configuration.

Chapter 3 — Setting Up the Development Environment

Required Tools

- Java JDK 17+
- IntelliJ IDEA or VS Code
- Maven or Gradle
- Postman
- MySQL or PostgreSQL

Download Java

You can download Java from:

[Oracle Java Official Website](#)

Generate a Spring Boot Project

Use:

[Spring Initializr](#)

Recommended Dependencies

- Spring Web

- Spring Data JPA
- MySQL Driver
- Spring Security
- Lombok

Chapter 4 — Creating Your First Spring Boot Application

Main Application Class

```
package com.example.demo;
```

```
import org.springframework.boot.SpringApplication;  
import  
org.springframework.boot.autoconfigure.SpringBootApplication;
```

```
@SpringBootApplication  
public class DemoApplication {  
  
    public static void main(String[] args) {  
        SpringApplication.run(DemoApplication.class, args);  
    }  
}
```

Explanation

@SpringBootApplication enables:

- component scanning,
- auto configuration,
- Spring Boot setup.

Running the Application

```
mvn spring-boot:run
```

Chapter 5 — Understanding Project Structure

Typical Spring Boot project structure:

```
src/  
├── main/  
│   ├── java/  
│   └── resources/  
└── test/
```

Important Folders

Folder	Purpose
controller	Handles HTTP requests
service	Business logic
repository	Database access
entity	Database models

Chapter 6 — Dependency Injection and Beans

Spring manages objects automatically using Dependency Injection.

Creating a Service

```
import org.springframework.stereotype.Service;
```

```
@Service
```

```
public class BookService {  
  
    public String getMessage() {  
        return "Spring Boot Service";  
    }  
}
```

Injecting a Service

```
import  
org.springframework.web.bind.annotation.GetMapping;  
import  
org.springframework.web.bind.annotation.RestController;
```

```

@RestController
public class BookController {

    private final BookService bookService;

    public BookController(BookService bookService) {
        this.bookService = bookService;
    }

    @GetMapping("/message")
    public String message() {
        return bookService.getMessage();
    }
}

```

Chapter 7 — Building REST APIs

REST APIs allow frontend and backend communication.

Simple REST Endpoint

```

@RestController
@RequestMapping("/api/books")
public class BookController {

    @GetMapping
    public String books() {
        return "List of books";
    }
}

```

HTTP Methods

Method	Purpose
--------	---------

GET	Retrieve data
POST	Create data
PUT	Update data
DELETE	Remove data

Chapter 8 — Working with Databases

Spring Boot integrates easily with databases.

application.properties

spring.datasource.url=jdbc:mysql://localhost:3306/library

spring.datasource.username=root

spring.datasource.password=password

Chapter 9 — Spring Data JPA

JPA simplifies database operations.

Creating an Entity

import jakarta.persistence.*;

@Entity

public class Book {

@Id

**@GeneratedValue(strategy =
GenerationType.IDENTITY)**

private Long id;

private String title;

}

Creating a Repository

```
import
org.springframework.data.jpa.repository.JpaRepository;

public interface BookRepository extends
JpaRepository<Book, Long> {
}
```

Chapter 10 — Validation and Exception Handling

Validation ensures data correctness.

Request Validation

```
@NotBlank
private String title;
```

Global Exception Handler

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(Exception.class)
    public String handleException(Exception ex) {
        return ex.getMessage();
    }
}
```

Chapter 11 — Spring Boot Security Basics

Security protects applications from unauthorized access.

Adding Spring Security

```
<dependency>
    <groupId>org.springframework.boot</groupId>
```

```
<artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

Features

- Authentication
- Authorization
- Password encryption
- JWT support

Chapter 12 — Configuration and Properties

Spring Boot uses configuration files.

application.properties Example

```
server.port=8081
spring.application.name=library-api
```

Profiles

Profiles allow different environments:

- development
- testing
- production

Chapter 13 — Testing Spring Boot Applications

Testing improves software quality.

Unit Test Example

```
@SpringBootTest
class DemoApplicationTests {

    @Test
    void contextLoads() {
    }
}
```


Types of Testing

- Unit testing
- Integration testing
- API testing

Chapter 14 — Deploying Spring Boot Applications

Applications can be packaged as JAR files.

Build Project

mvn clean package

Run JAR File

java -jar demo.jar

Deployment Platforms

- Docker
- AWS
- Render
- Railway
- Heroku

Chapter 15 — Best Practices in Spring Boot

Organize Code Properly

Separate:

- controllers,
- services,
- repositories,
- entities.

Use DTOs

DTOs protect entities and improve API structure.

Handle Exceptions Globally

Centralized error handling improves maintainability.

Chapter 16 — Real-World Project Ideas

Beginner-Friendly Projects

1. Library Management System
2. Student Management API
3. Task Management Application
4. E-commerce Backend
5. Blog REST API

Intermediate Projects

- JWT Authentication System
- Online Banking API
- E-learning Platform
- Hotel Reservation System

Chapter 17 — Conclusion

Spring Boot is one of the most powerful frameworks for building Java applications quickly and efficiently.

By learning Spring Boot, developers can create:

- scalable backend systems,
- REST APIs,
- enterprise applications,
- cloud-native services.

The key to mastering Spring Boot is continuous practice and building real-world projects.

Final Advice

- Practice building APIs regularly
- Learn Spring Security deeply
- Understand JPA relationships
- Build complete backend projects
- Explore microservices architecture
- Read official Spring documentation

End

Introduction to Spring Boot — Beginner Practical Guide